Facultad de Informática

UNIVERSIDAD DE MURCIA

PRÁCTICA DE DIVIDE Y VENCERÁS Y ANÁLISIS DE ALGORITMOS

Profesor: JOAQUÍN CERVERA LÓPEZ

Nombre	Grupo	Subgrupo	e-mail	DNIs
María Almarcha Lorca	2	2.2	m.almarchalorca@um.es	34332923H
Fadwa Abbaq El Gharras	2	2.2	fadwa.a.g@um.es	30850969B

Problema asignado:

3) Dada una cadena A de longitud n, y un natural m <= n, encontrar la subcadena B de m caracteres con más caracteres consecutivos ordenados ascendentemente (incluyendo caracteres iguales). Llamenos C a la subcadena de B formada por dichos caracteres. Devolver como solución el índice p de comienzo B y el número de caracteres en C. En caso de empate, será válida cualquiera de las soluciones óptimas.

Ejemplo: n=10, m=5

A= c d d a b c d a c c

Solución: B, posición de inicio igual a 4, y número de caracteres en C igual a 4 (a b c d).

ÍNDICE

1. Diseño 2
2. Análisis teórico 7
3. Implementación 9
4. Validación 10
4.1 Casos prueba individuales 10
4.2 Casos aleatorios 10
5. Análisis experimental 11
6. Contraste 13
7. Conclusiones y valoraciones personales de la actividad 14

1. Diseño

1.1 Algoritmo iterativo

Para crear este algoritmo, hemos pensado en hacer dos bucles:

- Un **while** que recorra la cadena `A` de longitud `n`.
- Un **for** que recorra cada una de las subcadenas `B` posibles de longitud `m`.

El algoritmo recorre todas las posibles subcadenas de `m` caracteres que estén en la cadena `A`. Para cada subcadena, mira si el carácter actual y el siguiente están ordenados respectivamente. Si es así, actualiza el contador de carácteres ordenados y sigue avanzando. Si no, comprobamos si el contador actual es mayor o igual que el máximo encontrado hasta ese momento (`max`). Si lo es, actualizamos tanto `max` como `p`, que guarda la posición inicial de la subcadena.

Seguimos recorriendo la subcadena por si más adelante hay un conjunto ordenado más grande. Una vez que hemos terminado de recorrer la subcadena, hacemos una última comprobación del contador por si la secuencia ordenada más larga estaba al final y pasamos a la siguiente subcadena `B` posible.

Para hacerlo más eficiente, ya que el máximo de carácteres ordenados está acotado, hemos añadido una condición al bucle principal para que termine cuando los carácteres ordenados de una subcadena sean los máximos posibles (sean `m`), ya que no puede encontrarse una solución mejor.

Repetimos el proceso hasta que se han recorrido todas las posibles subcadenas o hasta que se alcanza esa cota máxima.

Como el algoritmo necesita proporcionar dos resultados (tanto índice `p` de comienzo de la subcadena como el número `max` de caracteres en `C`), hemos pasado por referencia la variable `p`, de modo que pueda actualizarse y conservar su valor al finalizar el algoritmo, y hemos devuelto la variable `max` como valor de retorno de la función.

Decisiones de diseño:

- Utilizamos la expresión `(i < n - m + 1)` en el bucle while para calcular directamente cuántas combinaciones de subcadenas de `m` caracteres se pueden hacer dentro de la cadena `A`.
- Utilizamos la expresión `(j ≤ i + m - 2)` en el bucle for para recorrer hasta el penúltimo carácter de la subcadena, ya que como lo vamos a comparar con su siguiente carácter (`A[j] ≤ A[j+1]`), no hace falta recorrer el último también.
- Inicializamos el contador y el máximo en 1, ya que como mínimo en toda subcadena siempre habrá un carácter ordenado (él mismo).
- Inicializamos `p` a uno, ya que en el peor de los casos la mejor solución es de longitud 1 empezando por la primera posición de la cadena. Al actualizarla, le pasamos el valor `i + 1` para que devuelva la posición de la cadena empezando por 1.

Como este algoritmo es la versión iterativa (por fuerza bruta comprobando todo paso a paso), no sigue el esquema de Divide y Vencerás y no es lo más eficiente posible, ya que no estamos partiendo el problema en trozos más pequeños.

El **pseudocódigo** de la función es el siguiente:

Algorithm 1 Subcadena con más caracteres ordenados (iterativo)

```
function ITERATIVO(A, n, m, p)
    if n < m then
        p ← 0
        return 0
    end if
    p ← 1
    max ← 1
    i ← 0
    while i < n − m + 1 y max < m do
        cont ← 1
        for j ← i hasta i + m − 2 do
            if A[j] ≤ A[j + 1] then
                cont ← cont + 1
            else
                if cont > max then
                    max ← cont
                    p ← i + 1
                end if
                cont ← 1
            end if
        end for
        if cont > max then
            max ← cont
            p ← i + 1
        end if
        i ← i + 1
    end while
    return max
end function
```

1.2 Algoritmo divide y vencerás

Para crear este algoritmo, nos hemos basado en el esquema de Divide y Vencerás. Al plantear el problema nos dimos cuenta de que lo difícil no era solo partir el array por la mitad y buscar en cada lado, sino que la mejor subcadena de tamaño `m` podría estar justo en medio, cruzando la frontera entre la mitad izquierda y la derecha.

Por tanto, hemos dividido el algoritmo de la siguiente forma:

- **Caso Base**: Lo primero es mirar el tamaño del tramo actual (`fin - ini + 1`). Si es menor que `m`, devolvemos 0 porque es imposible formar la subcadena. Si mide exactamente `m`, ya no dividimos más: saltamos a la función auxiliar que hemos hecho (funcion `solucionPequena`), donde hacemos un pequeño bucle (igual que en la versión iterativa) para ver la racha máxima de ese tramo concreto y actualizamos su posición inicial.
- **División**: Si el tramo es mayor que `m`, calculamos la mitad para partir el problema en dos trozos más pequeños.

- **Llamadas Recursivas:** Nos llamamos a nosotros mismos (la función `dyv`) una vez para la mitad izquierda y otra para la mitad derecha.
- **Combinación**: Para esto hemos usado una función a parte (función `combinar`), para que el código quede más legible. Esta función revisa únicamente las subcadenas de tamaño `m` que empiezan en la parte izquierda y terminan en la derecha (la zona de cruce). Luego simplemente comparamos quién tiene el récord: la izquierda, la derecha o el centro. Nos quedamos con el mayor máximo y actualizamos la variable `p`.

Decisiones de diseño:

- **Variables temporales para la referencia**: Como el enunciado pide devolver la posición `p` por referencia, hemos creado variables locales temporales (`pIzq` y `pDer`) que guardan la posición ganadora de cada mitad. Solo actualizamos la `p` original al final del todo, en la decisión final, cuando ya sabemos con certeza qué zona tiene el máximo global.
- **Cálculo seguro en la zona del cruce**: Para revisar las subcadenas que cruzan la mitad, tuvimos que calcular a mano desde (`inicio_cruce = mitad - m +2`) hasta (`fin_cruce = mitad`) dónde teníamos que mirar. Le añadimos unas condiciones `if` de seguridad para evitar salirnos de los límites del subvector actual (`ini` y `fin`). Así nos aseguramos de no acceder a memoria que no toca y evitamos errores de segmentación.
- **Gestión de empates**: A la hora de comparar los máximos de las tres zonas o dentro de los bucles, hemos usado el operador mayor (>). De esta forma, si hay varias subcadenas que empatan con la misma longitud máxima, solo se actualiza el contador si la supera, quedándose con la primera de las mejores opciones. Como el enunciado especifica explícitamente que "en caso de empate, será válida cualquiera de las soluciones óptimas", esto nos ahorra estar actualizando constantemente el máximo.
- **Estructura de datos sencillas**: Hemos usado tipos básicos primitivos (enteros para los contadores y las posiciones, y el array de caracteres original) para no sobrecargar el coste espacial del algoritmo ni reservar memoria extra innecesaria, aunque la razón más importante para nosotras es porque nos sentimos más cómodas trabajando con esos tipos.

El **pseudocódigo** de la función es el siguiente:

Algorithm 2 Esquema DyV adaptado

```
function DyV(A, ini, fin, m, p)
    longitud ← fin − ini + 1
    if longitud < m then
        return 0
    end if
    if longitud == m then
        return Solucion_Pequena(A, ini, fin, p)
    end if
    mitad ← (ini + fin) // 2
    pIzq ← 0
    maxIzq ← DyV(A, ini, mitad, m, pIzq)
    if maxIzq == m then
        p ← pIzq
        return maxIzq
    end if
    pDer ← 0
    maxDer ← DyV(A, mitad + 1, fin, m, pDer)
    if maxDer == m then
        p ← pDer
        return maxDer
    end if
    return Combinar(A, ini, fin, mitad, m, maxIzq, pIzq, maxDer, pDer, p)
end function
```

El Algoritmo 3 y el Algoritmo 4 contienen las definiciones de la función `solucionPequena` y la función `combinar` respectivamente. Un array es considerado pequeño si el tramo mide exactamente `m`.

Algorithm 3 Función SOLUCIÓNPEQUEÑA

```
function SolucionPequena(A, ini, fin, p)
    max_local ← 1
    cont ← 1
    for i ← ini hasta fin − 1 do
        if A[i] ≤ A[i + 1] then
            cont ← cont + 1
        else
            if cont > max_local then
                max_local ← cont
            end if
            cont ← 1
        end if
    end for
    if cont > max_local then
        max_local ← cont
    end if
    p ← ini + 1
    return max_local
end function
```

Algorithm 4 Función COMBINAR

```
function COMBINAR(A, ini, fin, mitad, m, maxIzq, pIzq, maxDer, pDer, p)
    maxCentro ← 0
    pCentro ← 0
    inicio_cruce ← máx(mitad − m + 2, ini)
    fin_cruce ← mín(mitad, fin − m + 1)
    for i ← inicio_cruce hasta fin_cruce do
        cont ← 1
        max_actual ← 1
        for j ← i hasta i + m − 2 do
            if A[j] ≤ A[j + 1] then
                cont ← cont + 1
            else
                if cont > max_actual then
                    max_actual ← cont
                end if
                cont ← 1
            end if
        end for
        if cont > max_actual then
            max_actual ← cont
        end if
        if max_actual > maxCentro then
            maxCentro ← max_actual
            pCentro ← i + 1
        end if
        if maxCentro == m then
            p ← pCentro
            return maxCentro
        end if
    end for
    max_final ← maxIzq
    p ← pIzq
    if maxCentro > max_final then
        max_final ← maxCentro
        p ← pCentro
    end if
    if maxDer > max_final then
        max_final ← maxDer
        p ← pDer
    end if
    return max_final
end function
```

Consideraciones importantes:

Se ha utilizado asistencia de IA exclusivamente para transcribir nuestro pseudocódigo al formato visual en LaTeX, para que sea más legible y ordenado, utilizando la página *overleaf.com*.

2. Análisis teórico

Nuestro algoritmo Divide y Vencerás tiene un tamaño de $n = fin - ini + 1$, al dividir el problema por la mitad, se crean dos subproblemas (izquierdo y derecho). El coste viene dado por la siguiente ecuación:

$$t(n) = t(n_1) + t(n_2) + f(n)$$

donde n_1 y n_2 son los tamaños del subproblema izquierdo y el subproblema derecho y $f(n)$ el de la combinación de enmedio. Al dividir el problema por la mitad utilizando $mitad = (ini + fin) / 2$, los tamaños de los subproblemas quedan como:

- ★ **Subproblema izquierdo:**
  $n_1 = mitad - ini + 1$
- ★ **Índices:**
  `[ini, mitad]`

- ★ **Subproblema derecho:**
  $n_2 = fin - mitad$
- ★ **Índices:**
  `[mitad + 1, fin]`

Para sacar n_1 y n_2, como sabemos que $n = fin - ini + 1 \rightarrow fin = n + ini - 1$ y $mitad = (ini + fin) / 2$ entonces podemos hacer:

$$mitad = \frac{ini + fin}{2} = \frac{ini + (n + ini - 1)}{2} = \frac{2\,ini + n - 1}{2} = ini + \frac{n-1}{2}$$

- Sustituimos el valor de $mitad$ en $n_1 = mitad - ini + 1$ para sacar n_1:

$$n_1 = \left(ini + \frac{n-1}{2}\right) - ini + 1 = \frac{n-1}{2} + 1 = \frac{n-1+2}{2} = \frac{n+1}{2} \simeq \frac{n}{2} \text{ superior}$$

- Sustituimos el valor de $mitad$ y de fin en $n_2 = fin - mitad$ para sacar n_2:

$$n_2 = (n + ini - 1) - \left(ini + \frac{n-1}{2}\right) = n - 1 - \frac{n-1}{2} = \frac{n-1}{2} \simeq \frac{n}{2} \text{ inferior}$$

La ecuación de los tiempos finalmente quedaría como:

$$t(n) = t\left(\frac{n}{2}\right) + t\left(\frac{n}{2}\right) + f(n)$$

Si suponemos que n es potencia de 2 ($n = 2^k$ para algún k entero), tenemos:

$$t(n) = 2t\left(\frac{n}{2}\right) + f(n)$$

Para la parte de $f(n)$, hay que calcular el tiempo que ocupan los cálculos de la función `combinar`. Esta función contiene dos bucles `for` (uno dentro de otro):

```
int mitad = (ini + fin) / 2;
int inicio_cruce = mitad - m + 2;
int fin_cruce = mitad;
```

```
for (int i = inicio_cruce; i<=fin_cruce; i++)
     for (int j = i; j < i+m-1; j++)
```

Como vemos, el número de iteraciones depende de `m` (el tamaño de la subcadena que buscamos). Como `m` es constante, no cambia cuando dividimos el array, así que el esfuerzo siempre es el mismo en cada recursión. Al ser independiente de `n`, su coste de tiempo es constante:

$$f(n) \in \Theta(1)$$

★ **Peor caso:** t_M

- **Entrada:** cadena `A` entera decreciente (ej: '`zyxwv`…').
- **Explicación:** como los elementos de la subcadena nunca crecen, el algoritmo nunca alcanzará el tamaño `m` y tendrá que dividir y recorrer todo el array.
- **Ecuación:** $t_M(n) = 2t_M\left(\frac{n}{2}\right) + \Theta(1)$
- **Teorema Maestro:** $t(n) = a\,t\left(\frac{n}{b}\right) + f(n)$

  Con $a = 2$, $b = 2$ y $c = 0$, se cumple que $log_2(2) = 1 > 0$. Por tanto, el tiempo es $t_M(n) \in \Theta(n^{log_2 2}) = t_M(n) \in \Theta(n)$.

★ **Mejor caso:** t_m

- **Entrada:** cadena `A` de caracteres idénticos (ej: '`aaaaa`…') o ascendente.
- **Explicación:** el algoritmo encuentra una subcadena de tamaño `m` donde todos los caracteres sean iguales o estén de manera ascendente. Como tenemos el máximo acotado a `m`, se detiene y no necesita seguir dividiendo ni recorriendo el resto de la cadena, sin necesidad de llamar a la mitad derecha ni a la combinación.
- **Ecuación:** $t_m(n) = t_m\left(\frac{n}{2}\right) + \Theta(1)$
- **Teorema Maestro:** $t(n) = a\,t\left(\frac{n}{b}\right) + f(n)$

  Con $a = 1$, $b = 2$ y $c = 0$, se cumple que $log_2(1) = 0 = 0$. Por tanto, el tiempo es $t_m(n) \in \Theta\left(n^{log_2 1} log\, n\right) = \Theta(1 \cdot log\, n) = \Theta(log\, n)$.

Si no suponemos que n es potencia de 2 y aplicamos el teorema de la b-armónica nos queda:

- $t(n)$ (con el techo y suelo) es eventualmente no decreciente.
- n y $log\, n$ son crecientes.
- n y $log\, n$ son 2-armónicas.

3. Implementación

Para que el proyecto quedara bien organizado, limpio y fácil de compilar, hemos estructurado el código en varios ficheros tal y como se sugería en las instrucciones:

- **`dyv.h`** y **`dyv.cpp`**: En la cabecera declaramos las firmas de las funciones, y en el .cpp está el código principal tanto del algoritmo iterativo como del de Divide y Vencerás. En ambos, hemos devuelto la racha máxima con un `return` y la posición inicial pasándola por referencia (`int &p`).

- **`tests_unitarios.cpp`** y **`test_oraculo.cpp`**: Nos sirvieron para validar el código. El de unitarios lo usamos primero con ejemplos a mano (como el del enunciado) para comprobar que ambos algoritmos funcionaban, más tarde añadimos más casos de prueba. El oráculo lo usamos para generar arrays aleatorios y comprobar que el `iterativo` y el `DyV` daban siempre el mismo resultado.

- **`tiempos.cpp`**: Es el programa que usamos para medir el coste empírico. Va generando arrays aleatorios de diferentes tamaños, mide el tiempo de ejecución y lo escribe en un csv.

- **`makefile`**: Hemos configurado este archivo para no tener que compilar a mano con `g++` todo el rato. Con un simple `make` en la terminal, automatizamos la creación de los ejecutables.
  Este archivo nos muestra las dependencias de los distintos ficheros. Para poder compilar el fichero **`regresion.py`** (`python3 regresion.py`), previamente tiene que haber sido ejecutado el fichero de **`tiempos.cpp`** (`./tiempos`). Por lo que, si ejecutas `make` en la terminal, nuestro **`makefile`** directamente compilará y ejecutará uno por uno los ficheros en orden, creando los pdfs y archivos correspondientes.

- **`regresion.py`**: Un pequeño script en Python que usamos para procesar el `CSV` y sacar la recta de regresión de los tiempos

4. Validación

4.1 Casos prueba individuales

Hemos planteado a modo de ejemplo los siguientes casos de prueba individuales:

Mayor subcadena al principio:

Entrada: `n = 8  m = 4  A[n] = {a, b, c, d, z, y, x, w}`
Solución: `B[m] = {a, b, c, d}  p = 1  C[] = {a, b, c, d}  (Lon = 4)`

Mayor subcadena al final:

Entrada: `n = 8  m = 4  A[n] = {z, x, x, w, a, b, c, a}`
Solución: `B[m] = {w, a, b, c}  p = 4  C[] = {a, b, c}  (Lon = 3)`

Mayor subcadena en la mitad:

Entrada: `n = 8  m = 4  A[n] = {z, y, a, b, c, d, x, w}`
Solución: `B[m] = {a, b, c, d}  p = 3  C[] = {a, b, c, d}  (Lon = 4)`

Mejor caso (Constante):

Entrada: `n = 8  m = 4  A[n] = {a, a, a, a, a, a, a, a}`
Solución: `B[m] = {a, a, a, a}  p = 1  C[] = {a, a, a, a}  (Lon = 4)`

Mejor Caso (Creciente):

Entrada: `n = 8  m = 4  A[n] = {a, b, c, d, e, f, g, h}`
Solución: `B[m] = {a, b, c, d}  p = 1  C[] = {a, b, c, d}  (Lon = 4)`

Peor caso (Decreciente):

Entrada: `n = 8  m = 4  A[n] = {z, y, x, w, v, u, t, s}`
Solución: `B[m] = {z, y, x, w}  p = 1  C[] = {z}  (Lon = 1)`

4.2 Casos aleatorios

Para llevar a cabo la validación del algoritmo con casos aleatorios hemos creado el programa **`test_oraculo.cpp`**. En este programa generamos secuencias aleatorias formadas por letras de la ‘`a`’ a la ‘`z`’ con longitudes de `5, 10, 20, 50, 100` y `1000` caracteres. Para cada una de ellas, se ejecutan los dos algoritmos (`iterativo` y `dyv`) buscando la mejor subcadena de tamaño `m`.

Finalmente, comprobamos que el resultado de ambos algoritmos coincide. Si todos los resultados coinciden, imprimimos un mensaje indicando que todas las pruebas se han superado.

5. Análisis experimental

Para comprobar cómo se comporta el algoritmo en realidad, hemos medido su tiempo de ejecución con cadenas de 1000 hasta más 1 000 000 de caracteres, duplicando su tamaño en cada paso. Cogemos 10 medidas por cada tamaño y calculamos la mediana.

Diseñamos estas dos funciones para que generen el mejor y peor caso, con los que veremos como se comporta nuestro algoritmo:

- **El peor caso**: se genera creando una cadena con todos los carácteres de manera decreciente (ej: '`zyxwv`'). Como los elementos de la subcadena nunca crecen, el algoritmo nunca alcanzará el tamaño `m` y tendrá que dividir y recorrer todo el array.

Algorithm 5 Generar peor caso

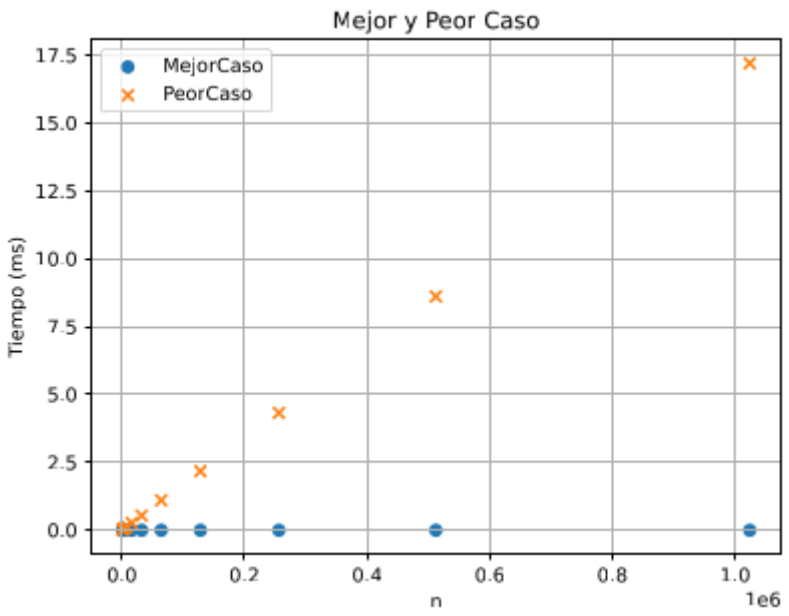
```
function GENERAR_PEOR_CASO(longitud)
    crear vector A de tamaño longitud
    for i ← 0 hasta longitud − 1 do
        A[i] ← 'z' − (i (mód 26))
    end for
    return A
end function
```

- **El mejor caso:** se genera creando una cadena donde todos sus carácteres sean el mismo (ej: '`aaaaa`') (o ascendente ej: '`abcde`'). El algoritmo encuentra una subcadena de tamaño `m` donde todos los caracteres están ordenados. Como tenemos el máximo acotado a `m`, se detiene y no necesita seguir dividiendo ni recorriendo el resto de la cadena.

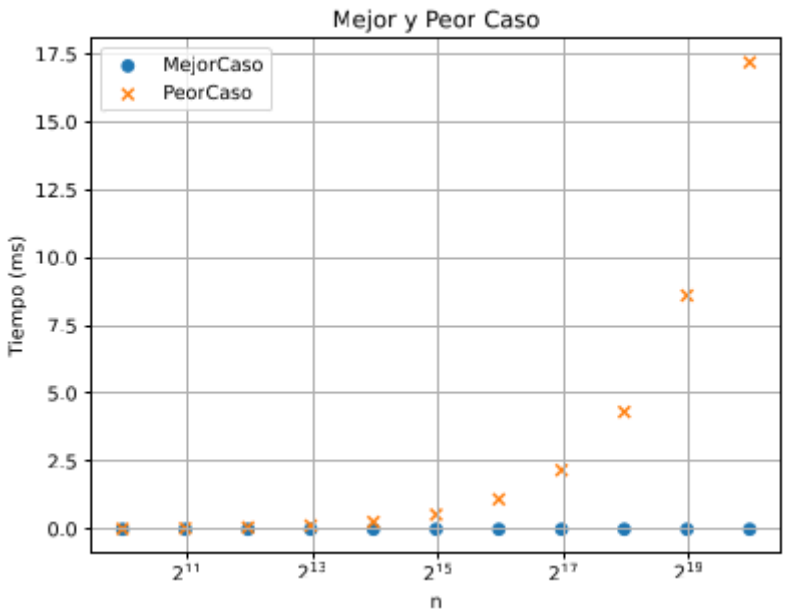
Algorithm 6 Generar mejor caso

```
function GENERAR_MEJOR_CASO(longitud)
    crear vector A de tamaño longitud
    for i ← 0 hasta longitud − 1 do
        A[i] ← 'a'                    ▷ También valdría 'a' + (i mod 26)
    end for
    return A
end function
```

La siguiente figura muestra las medidas de tiempo en ms para cada tamaño de entrada `n` y cada caso.



La siguiente figura es como la anterior, pero usando la escala logarítmica en el eje x.



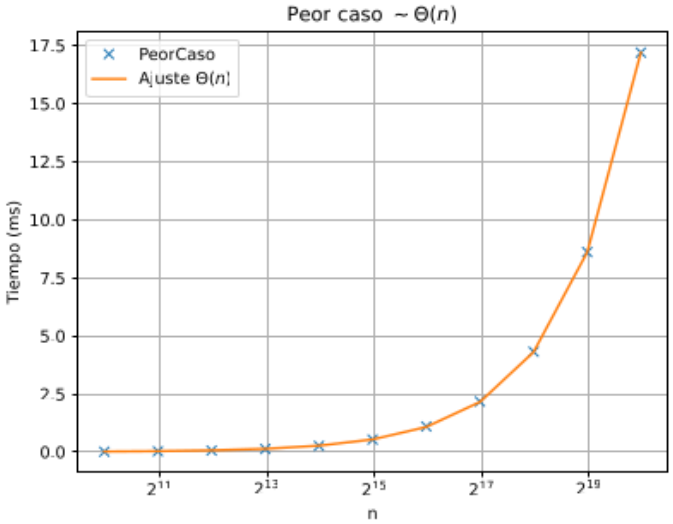
6. Contraste

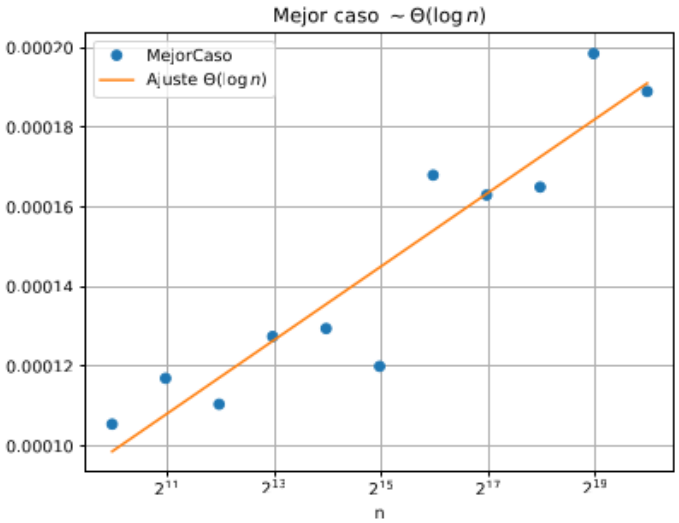
Para ver si nuestros cálculos teóricos coincidían con lo que pasa de verdad en el ordenador, cogimos los datos de nuestro archivo `resultados.csv`, le aplicamos el script de regresión y vimos la gráfica (que se adjunta a continuación).

En el peor caso, usando la ecuación de recurrencia y el Teorema Maestro habíamos calculado que el coste era lineal, $O(n)$. Y efectivamente, al ver la nube de puntos en la gráfica generada por `tiempos.cpp`, sale una línea recta en diagonal a medida que hacemos la cadena más grande. Y al hacer el ajuste, el valor R^2 nos ha dado 0.99999. Al estar tan cerca del 1, nos confirma nuestra teoría, el tiempo crece de forma proporcional a la cantidad de datos.

Por otro lado, en el mejor caso, nuestros cálculos anteriores decían que el coste iba a ser constante, O(1), porque el código frena enseguida y no llega a hacer toda la recursión. Al mirar la gráfica, se ve muy claro: una línea recta cuyos datos tardan poquísimo, unos 0.0004 ms. Da igual si le metemos una cadena gigante, el tiempo se mantiene estable.

En conclusión, los datos de las gráficas nos confirman al 100% que los tiempos que calculamos teóricamente son correctos y no hay discrepancias raras





7. Conclusiones y valoraciones personales de la actividad

En total, completar esta práctica nos ha llevado un tiempo estimado de unas 25 horas de trabajo en grupo. Tiempo invertido tanto en clase como en llamadas de Discord e individualmente.

La parte de algoritmo iterativo fue bastante directa de programar, pero enfocar el de Divide y Vencerás nos costó un poquito más. Hicimos varios cambios desde que entregamos la primera tarea de cómo iba el proyecto hasta ahora, como dividir la función de **dyv** en varias funciones para que el código fuese mucho más legible.

El mayor reto al que nos enfrentamos fue plantear las matemáticas de la zona de cruce, concretamente, pensar que el índice `inicio_cruce` tenía que ser mitad `- m + 2` y ponerle los `if` de seguridad para no acabar haciendo una segmentación al salirnos de los límites del array en memoria.

Sin embargo, hacer esta práctica nos ha resultado súper útil y de lo más gratificante. Ver que un algoritmo recursivo que parece tan abstracto saca exactamente lo mismo que uno de fuerza bruta da mucha satisfacción. Además, el hecho de medir los tiempos reales en milisegundos y comprobar que la gráfica se alinea a la perfección con la complejidad $O(n)$ que vimos en clase, hace que todo el tema de la eficiencia, los sumatorios y el Teorema Maestro cobre mucho más sentido para nosotras.

También decir que hemos resuelto el problema de manera que, si hay varias subcadenas que pueden ser las candidatas de la solución y todas son igual de válidas, siempre se devuelva la que se encuentra primera de ellas. Para esto hemos actualizado los contadores de carácteres ordenados sólo cuando alguna cadena supera la actual, no cuando lo iguala.

Recalcar que hemos aprendido a hacer un diagrama de puntos en geogebra por nosotras mismas, aunque finalmente nos quedamos con el archivo `.py` que lo generaba específicamente como nos lo pedía la práctica.

Y, aunque no hemos generado nuestro archivo `makefile` con IA, sí que nos hemos ayudado de ella a la hora de saber cuantos archivos `.o` había que generar y cómo limpiar todo correctamente.